

WIFI

```
#define BLYNK_TEMPLATE_ID   "TMPL6aEV5Tzyf"
#define BLYNK_TEMPLATE_NAME "Smart Pet Tracker"
#define BLYNK_AUTH_TOKEN    "GFrp0mSW2vGnPnLvq-D1b_NbcksFxFzT"
```

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <TinyGPS++.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiClientSecure.h>
#include <BlynkSimpleEsp32.h>
```

```
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
```

```
// 🎯 ตั้งค่าพินตามที่ต่อจริงหน้างาน
```

```
#define GPS_RX_PIN 21
#define GPS_TX_PIN 20
#define I2C_SDA_PIN 8
#define I2C_SCL_PIN 9
```

```
const char* ssid = "WJNK";
const char* password = "12345678";
```

```
const String LINE_TOKEN =
"QCnqt656xzGsM3da7GlsgFd5R1otHqbVncg17unluhD9UvO0bzcS53m+tNhi+OcOVKEjUKf73iSE
```

NjbzZGjl6X9tKbZK4AYPb0o0kvQleYaPMZt2ySPNDwhDt6U1APj3ikBub5eVLZPgPyPb8DwWqgdB0
4t89/1O/w1cDnyilFU=";

const String LINE_USER_ID = "U7d93b01ef2609846c1efc716b4c492ca";

Adafruit_MPU6050 mpu;

TinyGPSPlus gps;

HardwareSerial SerialGPS(1);

bool isGyroConnected = false;

unsigned long lastDisplayTime = 0;

unsigned long lastBlynkStreamTime = 0;

unsigned long lastSensorReadTime = 0;

unsigned long gpsByteCounter = 0;

float current_total_acc = 0.0;

double homeLat = 0.0;

double homeLng = 0.0;

bool isHomeSet = false;

double lastValidLat = 18.802500;

double lastValidLng = 98.951500;

bool hasValidGpsOnce = false;

//  ตัวแปรสำหรับระบบคำนวณ Packet Loss / Success Rate

unsigned long totalExpectedPackets = 0;

unsigned long totalSentPackets = 0;

unsigned long totalFailedPackets = 0;

unsigned long lastPacketMeasureTime = 0;

```

#define VPIN_GYRO      V1
#define VPIN_MAP_TEXT  V2

#define VPIN_STATUS    V3 // 🟢 รวมแสดงผล: [โหมด] + สถานะการเคลื่อนไหว + สถานะรั้ว
#define VPIN_SETHOME   V4
#define VPIN_REFRESH   V5

#define VPIN_MAP_WEBLINK V11 // 🟢 Webpage Button ลิงก์แผนที่แบบอัปเดตอัตโนมัติ
#define VPIN_RADIUS     V7
#define VPIN_IMPACT_LIMIT V8
#define VPIN_HOME_COORD V9
#define VPIN_CLEAR_HOME V10

#define VPIN_BAT_LEVEL  V12 // 🔋 แถบแสดง % แบตเตอรี่ (Level Bar)
#define VPIN_BAT_REMAIN V13 // ⌚ ตัวแสดงเวลาคงเหลือ (Value Display)

double maxAllowedRadius = 3.0;
float gyroThreshold = 15.0;

String motionStatus = " 🟢 ปกติดี";
unsigned long highAccStartTime = 0;
bool isHighAccContinuous = false;
unsigned long lastImpactTime = 0;

// =====
// ⚙️ ADAPTIVE OPERATION MODE CONFIG (ระบบ 2 โหมดอัตโนมัติ)
// =====
unsigned long blynkStreamInterval = 15000; // ค่าเริ่มต้นโหมดปกติ: สตรีมทุกๆ 15 วินาที
String currentModeStr = "[ 🟢 โหมดปกติ - 15s]";
unsigned long emergencyTriggerTime = 0; // เวลาตรวจพบเหตุฉุกเฉินล่าสุด

// =====

```

```
//  VIRTUAL FUEL GAUGE CONFIG (อิงข้อมูลวัดกระแสจริงทำงาน)

// =====

const float USABLE_CAPACITY_MAH = 850.0; // แบต 1,000 mAh ตัดช่วง Step-up ดับ (~3.0V)
เหลือใช้งาน ~850 mAh

const float BAT_DRAIN_NORMAL = 172.3; // กระแสเฉลี่ยที่ดึงจากขั้วแบตเตอรี่: โหมดปกติ 15s
(คำนวณจาก Idle 107.7mA / Peak 136.8mA)

const float BAT_DRAIN_EMERGENCY = 179.4; // กระแสเฉลี่ยที่ดึงจากขั้วแบตเตอรี่: โหมดฉุกเฉิน 2s

float currentBatDrainMA = BAT_DRAIN_NORMAL;

unsigned long lastPowerCalcTime = 0;

float remainingCapacity = USABLE_CAPACITY_MAH;

int batteryPercent = 100;

float remainingHours = 4.93; // 850 / 172.3 = 4.93 ชม.

void updateVirtualBattery() {
    unsigned long now = millis();
    if (lastPowerCalcTime == 0) {
        lastPowerCalcTime = now;
        return;
    }

    // สลับค่าอัตราการดึงกระแสตามโหมดที่กำลังทำงานจริง
    currentBatDrainMA = (blynkStreamInterval <= 2000) ? BAT_DRAIN_EMERGENCY :
    BAT_DRAIN_NORMAL;

    // เวลาที่ผ่านไปนับเป็นชั่วโมง
    float elapsedHours = (float)(now - lastPowerCalcTime) / 3600000.0f;
    lastPowerCalcTime = now;

    // หักลบความจุสะสม (mAh = mA * hours)
    remainingCapacity -= (currentBatDrainMA * elapsedHours);
}
```

```

if (remainingCapacity < 0.0f) remainingCapacity = 0.0f;

// คำนวณ % คงเหลือ และชั่วโมงใช้งานที่เหลือ
batteryPercent = (int)((remainingCapacity / USABLE_CAPACITY_MAH) * 100.0f);
batteryPercent = constrain(batteryPercent, 0, 100);

remainingHours = remainingCapacity / currentBatDrainMA;
if (remainingHours < 0.0f) remainingHours = 0.0f;
}

void sendLineMessage(String message) {
    if (WiFi.status() == WL_CONNECTED) {
        WiFiClientSecure client;
        client.setInsecure();
        if (client.connect("api.line.me", 443)) {
            String payload = "{\"to\":\"" + LINE_USER_ID +
"\",\"messages\": [{\"type\":\"text\", \"text\":\"" + message + "\"}]}";
            client.println("POST /v2/bot/message/push HTTP/1.1");
            client.println("Host: api.line.me");
            client.println("Authorization: Bearer " + LINE_TOKEN);
            client.println("Content-Type: application/json");
            client.print("Content-Length: ");
            client.println(payload.length());
            client.println();
            client.println(payload);
            client.stop();
        }
    }
}

```

```

double calculateDistance(double lat1, double lng1, double lat2, double lng2) {
    double dLat = (lat2 - lat1) * M_PI / 180.0;
    double dLng = (lng2 - lng1) * M_PI / 180.0;
    double a = sin(dLat / 2) * sin(dLat / 2) +
        cos(lat1 * M_PI / 180.0) * cos(lat2 * M_PI / 180.0) *
        sin(dLng / 2) * sin(dLng / 2);
    double c = 2 * atan2(sqrt(a), sqrt(1 - a));
    return 6371000.0 * c;
}

```

```

BLYNK_CONNECTED() {
    Blynk.syncVirtual(VPIN_RADIUS);
    Blynk.syncVirtual(VPIN_IMPACT_LIMIT);
}

```

```

BLYNK_WRITE(VPIN_RADIUS) { maxAllowedRadius = param.asDouble(); }
BLYNK_WRITE(VPIN_IMPACT_LIMIT) { gyroThreshold = param.asFloat(); }

```

```

BLYNK_WRITE(VPIN_SETHOME) {
    if (param.asInt() == 1) {
        if (hasValidGpsOnce) {
            homeLat = lastValidLat;
            homeLng = lastValidLng;
            isHomeSet = true;

            Blynk.virtualWrite(VPIN_STATUS, currentModeStr + " 🏠 ตั้งพิกัด Home สำเร็จแล้ว!");
            Blynk.virtualWrite(VPIN_HOME_COORD, String(homeLat, 6) + "," + String(homeLng, 6));
        } else {
            Blynk.virtualWrite(VPIN_STATUS, currentModeStr + " ⚠️ ยังไม่พบสัญญาณ GPS จริง");
        }
    }
}

```

```
}
```

//  ปุ่ม Quick Refresh สั่งอัปเดต Webpage Button ผ่าน setProperty ด้าน

```
BLYNK_WRITE(VPIN_REFRESH) {
```

```
    if (param.asInt() == 1) {
```

```
        if (hasValidGpsOnce) {
```

```
            String quickLoc = String(lastValidLat, 6) + "," + String(lastValidLng, 6);
```

```
            String dynamicWebLink = "https://maps.google.com/?q=" + quickLoc + "&t=" +
```

```
String(millis());
```

```
            Blynk.virtualWrite(VPIN_MAP_TEXT, quickLoc);
```

```
            Blynk.setProperty(VPIN_MAP_WEBLINK, "url", dynamicWebLink);
```

```
            Blynk.virtualWrite(VPIN_STATUS, currentModeStr + "  บังคับอัปเดตพิกัดเข้าปุ่มแผนที่  
แล้ว!");
```

```
        } else {
```

```
            Blynk.virtualWrite(VPIN_STATUS, currentModeStr + "  กำลังรอสัญญาณ GPS...");
```

```
        }
```

```
    }
```

```
}
```

```
BLYNK_WRITE(VPIN_CLEAR_HOME) {
```

```
    if (param.asInt() == 1) {
```

```
        homeLat = 0.0;
```

```
        homeLng = 0.0;
```

```
        isHomeSet = false;
```

```
        Blynk.virtualWrite(VPIN_STATUS, currentModeStr + "  ยกเลิกและล้างพิกัด Home  
เรียบร้อยแล้ว!");
```

```
        Blynk.virtualWrite(VPIN_HOME_COORD, "0.000000,0.000000");
```

```
    }
```

```
}
```

```

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    WiFi.mode(WIFI_STA);

    btStop();

    delay(500);

    Serial.begin(115200);

    delay(1000);

    Serial.println("\n 🔍 =====");

    Serial.println(" 🛠️ [PIN DIAGNOSTIC CHECK]: ตรวจสอบขาฮาร์ดแวร์ทำงาน...");

    Serial.printf(" 📶 [GPS SERIAL1]: RX Pin = %d, TX Pin = %d\n", GPS_RX_PIN, GPS_TX_PIN);

    Serial.printf(" 📏 [I2C MPU6050]: SDA Pin = %d, SCL Pin = %d\n", I2C_SDA_PIN,
I2C_SCL_PIN);

    Serial.println("=====");

    SerialGPS.begin(9600, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);

    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    if (mpu.begin(0x68, &Wire, 0)) {
        isGyroConnected = true;

        Serial.println(" 🟢 [HW CHECK]: MPU6050 Gyro เชื่อมต่อสำเร็จผ่าน I2C!");
    } else {
        isGyroConnected = false;

        Serial.println(" 🚫 [HW CHECK ERROR]: ไม่พบ MPU6050!");
    }

    WiFi.setSleep(false);

```

```

WiFi.begin(ssid, password);
Blynk.config(BLYNK_AUTH_TOKEN);

lastPowerCalcTime = millis();
}

void loop() {
  unsigned long currentMillis = millis();

  if (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(ssid, password);
    int retryCount = 0;
    while (WiFi.status() != WL_CONNECTED && retryCount < 5) {
      delay(500);
      retryCount++;
    }
  }

  if (WiFi.status() == WL_CONNECTED && !Blynk.connected()) {
    Blynk.connect();
  }

  if (Blynk.connected()) {
    Blynk.run();
  }

  while (SerialGPS.available() > 0) {
    char c = SerialGPS.read();
    gps.encode(c);
    gpsByteCounter++;
  }
}

```

```
}
```

```
if (gps.location.isValid() && gps.location.isUpdated()) {
```

```
    lastValidLat = gps.location.lat();
```

```
    lastValidLng = gps.location.lng();
```

```
    hasValidGpsOnce = true;
```

```
}
```

```
//  ระบบคำนวณ Packet Success Rate ทุกๆ 5 วินาที
```

```
if (currentMillis - lastPacketMeasureTime >= 5000) {
```

```
    lastPacketMeasureTime = currentMillis;
```

```
    totalExpectedPackets++;
```

```
if (WiFi.status() == WL_CONNECTED && Blynk.connected()) {
```

```
    totalSentPackets++;
```

```
} else {
```

```
    totalFailedPackets++;
```

```
}
```

```
}
```

```
// อ่านค่าเซนเซอร์ความเร่งทุกๆ 1 วินาทีเพื่อตรวจจับพฤติกรรม
```

```
if (isGyroConnected && (currentMillis - lastSensorReadTime >= 1000)) {
```

```
    lastSensorReadTime = currentMillis;
```

```
    sensors_event_t a, g, temp;
```

```
    if (mpu.getEvent(&a, &g, &temp)) {
```

```
        current_total_acc = sqrtf(powf(a.acceleration.x, 2.0f) + powf(a.acceleration.y, 2.0f) +  
powf(a.acceleration.z, 2.0f));
```

```
if (current_total_acc > 25.0f) {
```

```
    emergencyTriggerTime = currentMillis;
```

```

motionStatus = " 🌟 บาดเจ็บ/ล้มคว่ำ!";
lastImpactTime = currentMillis;
static unsigned long lastNotifyTime = 0;
if (currentMillis - lastNotifyTime > 15000) {
    lastNotifyTime = currentMillis;

    Blynk.logEvent("impact_alert", " 🚨 เตือนภัย! ตรวจพบแรงกระแทกหรือสัตว์เลี้ยงล้ม
รุนแรงเฉียบพลัน!");

    sendLineMessage(" 🚨 [Wi-Fi Pet] เตือนภัย! ตรวจพบแรงกระแทกหรือสัตว์เลี้ยงล้ม
รุนแรงเฉียบพลัน!");
}

} else if (current_total_acc > gyroThreshold) {
    emergencyTriggerTime = currentMillis;
    if (highAccStartTime == 0) highAccStartTime = currentMillis;
    if ((currentMillis - highAccStartTime > 2500) && !isHighAccContinuous) {
        isHighAccContinuous = true;
        motionStatus = " ⚠️ วิ่งเร็ว/ตื่นตระหนก";
        Blynk.logEvent("impact_alert", " ⚠️ แจ้งเตือน: สัตว์เลี้ยงมีพฤติกรรมอัตราเร่งสูง
ต่อเนื่องผิดปกติ!");

        sendLineMessage(" 🚨 [Wi-Fi Pet] แจ้งเตือน: สัตว์เลี้ยงมีพฤติกรรมอัตราเร่งสูงต่อเนื่อง
ผิดปกติ!");
    }
} else {
    highAccStartTime = 0;
    isHighAccContinuous = false;

    if (currentMillis - lastImpactTime > 6000) motionStatus = " 🟢 ปกติดี";
}
}

// ตรวจสอบสถานะการหลุดรื้อบ้านเพื่อช่วยตัดสินใจเปลี่ยนโหมด

```

```

bool isEscaped = false;
if (isHomeSet && hasValidGpsOnce) {
    double dist = calculateDistance(lastValidLat, lastValidLng, homeLat, homeLng);
    if (dist > maxAllowedRadius) isEscaped = true;
}

// 🚨 จัดการโหมตและคาบเวลาสตรีม (Adaptive Stream Interval)
if (isEscaped || (emergencyTriggerTime > 0 && (currentMillis - emergencyTriggerTime <
8000))) {
    blynkStreamInterval = 2000; // โหมตฉุกเฉิน: เร่งสตรีมทุกๆ 2 วินาที
    currentModeStr = "[ 🚨 โหมตฉุกเฉิน - 2s]";
} else {
    blynkStreamInterval = 15000; // โหมตปกติ: ส่งทุกๆ 15 วินาที
    currentModeStr = "[ 🟢 โหมตปกติ - 15s]";
}
}

// 📶 คำนวณพลังงานและส่งข้อมูลขึ้น Blynk ตามรอบเวลาของโหมตที่กำลังทำงาน
if (Blynk.connected() && (currentMillis - lastBlynkStreamTime >= blynkStreamInterval)) {
    lastBlynkStreamTime = currentMillis;

    // อัปเดตการหักลบประจุตามอัตรากินกระแสจริง
    updateVirtualBattery();

    Blynk.virtualWrite(VPIN_GYRO, current_total_acc);

    String geoStatus = "";
    if (hasValidGpsOnce) {
        String googleMapsCoord = String(lastValidLat, 6) + "," + String(lastValidLng, 6);
        Blynk.virtualWrite(VPIN_MAP_TEXT, googleMapsCoord);
    }
}

```

```

// 🟢 อัปเดต URL ของ Webpage Button (V11) อัตโนมัติ

String dynamicWebLink = "https://maps.google.com/?q=" + googleMapsCoord +
"&t=" + String(currentMillis);

Blynk.setProperty(VPIN_MAP_WEBLINK, "url", dynamicWebLink);

if (isHomeSet) {
    double distance = calculateDistance(lastValidLat, lastValidLng, homeLat,
homeLng);
    if (distance > maxAllowedRadius) {
        geoStatus = " 🚨 หนีเที่ยว! (" + String(distance, 1) + " ม.);
        static unsigned long lastGeoTime = 0;
        if (currentMillis - lastGeoTime > 30000) {
            lastGeoTime = currentMillis;

            Blynk.logEvent("impact_alert", " 🚨 สัตว์เลี้ยงแอบหนีเที่ยวออกจากขอบเขตบริเวณ
บ้านแล้ว!");

            sendLineMessage(" 🚨 [Wi-Fi Pet] สัตว์เลี้ยงแอบหนีเที่ยวออกจากขอบเขตบ้านแล้ว!
พิกัด: https://maps.google.com/?q=" + googleMapsCoord);
        }
    } else {
        geoStatus = " 🏠 ในเขตรั้ว (" + String(distance, 1) + " ม.);
    }
} else {
    geoStatus = " 📍 รอตั้งโฮมทำงาน";
}
} else {
    geoStatus = " 🟡 กำลังค้นหาสัญญาณ GPS จริง...";
}

```

```

// 🌀 ส่งสถานะรวมพร้อมระบุโหมดชัดเจนเข้าหา V3
Blynk.virtualWrite(VPIN_STATUS, currentModeStr + " " + motionStatus + " | " +
geoStatus);

// 🔋 ส่งสถานะแบตเตอรี่เข้าหา V12 และ V13
Blynk.virtualWrite(VPIN_BAT_LEVEL, batteryPercent);
Blynk.virtualWrite(VPIN_BAT_REMAIN, String(remainingHours, 1) + " ชม.");
}

if (currentMillis - lastDisplayTime >= 2000) {
    lastDisplayTime = currentMillis;

    double currentDistance = 0.0;
    if (isHomeSet && hasValidGpsOnce) {
        currentDistance = calculateDistance(lastValidLat, lastValidLng, homeLat, homeLng);
    }

    long rssi = WiFi.RSSI();

    float successRate = (totalExpectedPackets > 0) ? ((float)totalSentPackets /
totalExpectedPackets) * 100.0 : 0.0;

    Serial.println("\n 📶 ===== [ แดชบอร์ดทดสอบโมเดล Wi-Fi ตรง ] =====");
    Serial.print(" 📶 [WIFI / BLYNK]: ");

    Serial.println(Blynk.connected() ? " 🟢 CONNECTED TO CLOUD SUCCESS!" : " ❌
DISCONNECTED (รอซิงค์...)");

    Serial.printf(" ⚙️ [OPERATION MODE]: %s\n", currentModeStr.c_str());

    Serial.printf(" 🔋 [BATTERY MONITOR]: %d%% | ⌚ คงเหลือ: %.2f ชม. (ตั้งกระแสเฉลี่ย:
%.1f mA)\n",
        batteryPercent, remainingHours, currentBatDrainMA);

```

```

Serial.printf(" 📶 [SIGNAL RSSI]: %ld dBm (%s)\n", rssi, (rssi > -75) ? " 🟢 สัญญาณดี" :
" 🟡 สัญญาณเริ่มอ่อน");

Serial.printf(" 📏 [DISTANCE TEST]: ห่างจากจุดตั้งต้น = %.1f เมตร\n", currentDistance);

Serial.printf(" 📦 [PACKET STATS]: ส่งสำเร็จ = %lu | หล่นหาย = %lu | Success Rate =
%.1f%%\n",

totalSentPackets, totalFailedPackets, successRate);

Serial.print(" 🧠 [STATEMACHINE]: " + motionStatus + "\n");

if (isGyroConnected) {

Serial.printf(" ✅ [GYRO]: ค่าความเร่งรวม = %.2f m/s²\n", current_total_acc);

}

if (hasValidGpsOnce) {

Serial.printf(" 🌐 [LIVE COORD]: Lat=%.6f, Lng=%.6f (HomeSet: %s)\n", lastValidLat,
lastValidLng, isHomeSet ? "YES" : "NO");

} else {

Serial.println(" 🌐 [LIVE COORD]: 🟡 กำลังรอจับสัญญาณดาวเทียม...");

}

Serial.println("=====");

}

}

```


LORAAa ส่ง

```
#include <Arduino.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_MPU6050.h>
```

```
#include <Adafruit_Sensor.h>
```

```
#include <TinyGPS++.h>
```

```
#include <WiFi.h>
```

```
#include "soc/soc.h"
```

```
#include "soc/rtc_cntl_reg.h"
```

```
#define LORA_M0 18
```

```
#define LORA_M1 19
```

```
#define LORA_AUX 23
```

```
#define LORA_RX_PIN 16
#define LORA_TX_PIN 17
#define LORA_SERIAL Serial2
```

```
#define GPS_RX_PIN 4
#define GPS_TX_PIN 5
```

```
#define I2C_SDA 21
#define I2C_SCL 22
```

```
Adafruit_MPU6050 mpu;
TinyGPSPlus gps;
```

```
bool isGyroConnected = false;
unsigned long lastLoRaSendTime = 0;
unsigned long lastSensorReadTime = 0;
unsigned long lastDisplayTime = 0;
unsigned long gpsByteCounter = 0;
unsigned long packetIdCounter = 0; // ตัวแปรนับลำดับแพ็กเกจสะสม
float current_total_acc = 0.0;
float last_acc_x = 0.0, last_acc_y = 0.0, last_acc_z = 0.0;
```

```
double lastValidLat = 18.805000;
double lastValidLng = 98.955000;
```

```
String assignedChannelStr = "UNKNOWN";
```

```
// =====
//  ADAPTIVE OPERATION MODE CONFIG (ระบบ 2 โหมดอัตโนมัติ)
// =====
```

```

unsigned long loraSendInterval = 15000; // ค่าเริ่มต้นโหมดปกติ: ส่งทุกๆ 15 วินาที

String currentModeStr = "  NORMAL (รอบส่ง 15s);

unsigned long emergencyTriggerTime = 0; // เวลาที่ตรวจพบเหตุฉุกเฉินล่าสุด

// =====
//  VIRTUAL FUEL GAUGE CONFIG (อิงข้อมูลวัดจริง: Idle 107.0mA / TX 168.5mA)
// =====

const float USABLE_CAPACITY_MAH = 850.0; // แบต 1,000 mAh หักตัดไฟ Step-up ดับเหลือใช้จริง
~850 mAh

const float BAT_DRAIN_NORMAL    = 171.0; // คำนวณจริงจากรอบ 15s
const float BAT_DRAIN_EMERGENCY = 177.3; // คำนวณจริงจากรอบ 2s

float currentBatDrainMA        = BAT_DRAIN_NORMAL;

unsigned long lastPowerCalcTime = 0;

float remainingCapacity        = USABLE_CAPACITY_MAH;

int  batteryPercent            = 100;

float remainingHours            = 4.97; // 850 / 171.0 = ~4.97 ชม.

void updateVirtualBattery() {
    unsigned long now = millis();
    if (lastPowerCalcTime == 0) {
        lastPowerCalcTime = now;
        return;
    }
}

//  สลับอัตราการดึงกระแสตามโหมดที่กำลังทำงานจริง
currentBatDrainMA = (loraSendInterval <= 2000) ? BAT_DRAIN_EMERGENCY :
BAT_DRAIN_NORMAL;

// เวลาที่ผ่านไปนับเป็นชั่วโมง
float elapsedHours = (float)(now - lastPowerCalcTime) / 3600000.0f;

```

```

lastPowerCalcTime = now;

// หักลบความจุสะสม (mAh = mA * hours)
remainingCapacity -= (currentBatDrainMA * elapsedHours);
if (remainingCapacity < 0.0f) remainingCapacity = 0.0f;

// คำนวณ % คงเหลือ และชั่วโมงใช้งานที่เหลือ
batteryPercent = (int)((remainingCapacity / USABLE_CAPACITY_MAH) * 100.0f);
batteryPercent = constrain(batteryPercent, 0, 100);

remainingHours = remainingCapacity / currentBatDrainMA;
if (remainingHours < 0.0f) remainingHours = 0.0f;
}

void configureLoraChannel() {
    digitalWrite(LORA_M0, LOW);
    digitalWrite(LORA_M1, HIGH);

    unsigned long modeSwitchStart = millis();
    while (digitalRead(LORA_AUX) == LOW && (millis() - modeSwitchStart < 1000)) {
        delay(5);
    }
    delay(100);

    while(LORA_SERIAL.available() > 0) { LORA_SERIAL.read(); }

    byte configCmd[] = {0xC0, 0x00, 0x06, 0x00, 0x00, 0x00, 0x62, 0x00, 0x49};
    LORA_SERIAL.write(configCmd, 9);
    LORA_SERIAL.flush();
}

```

```

unsigned long startTimeout = millis();

bool responseReceived = false;

while ((millis() - startTimeout < 500)) {
    if (LORA_SERIAL.available() > 0) {
        if (LORA_SERIAL.read() == 0xC1) {
            responseReceived = true;
            break;
        }
    }
    delay(10);
}

if (responseReceived) {
    assignedChannelStr = "CH 73 (ความถี่ 923.125MHz - 0x49) [  ACTIVE]";
    delay(20);
    while(LORA_SERIAL.available() > 0) { LORA_SERIAL.read(); }
} else {
    assignedChannelStr = "CH 73 (ความถี่ 923.125MHz) [  FORCED OVERRIDE]";
}

digitalWrite(LORA_M0, LOW);
digitalWrite(LORA_M1, LOW);
delay(100);
}

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
    WiFi.mode(WIFI_OFF);
    btStop();
}

```

```
delay(500);
```

```
Serial.begin(115200);
```

```
pinMode(LORA_M0, OUTPUT);
```

```
pinMode(LORA_M1, OUTPUT);
```

```
pinMode(LORA_AUX, INPUT);
```

```
unsigned long startWait = millis();
```

```
while (!Serial && (millis() - startWait < 4000)) { delay(10); }
```

```
Serial.println("\n 🚀 [RMUTL Tech] บอร์ดฝังส่ง ESP32 + Adaptive 2-Mode Power พร้อม  
ทำงาน!");
```

```
LORA_SERIAL.begin(9600, SERIAL_8N1, LORA_RX_PIN, LORA_TX_PIN);
```

```
configureLoraChannel();
```

```
Serial1.begin(9600, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);
```

```
Wire.begin(I2C_SDA, I2C_SCL);
```

```
if (mpu.begin(0x68, &Wire, 0)) {
```

```
    isGyroConnected = true;
```

```
    Serial.println(" 🟢 [MPU6050 SENSOR]: เชื่อมต่อเซนเซอร์ MPU6050 ผ่าน I2C (Address 0x68)  
สำเร็จ!");
```

```
    } else {
```

```
        Serial.println(" 🚫 [MPU6050 SENSOR ERROR]: ไม่พบเซนเซอร์ MPU6050! เช็กสาย SDA/SCL  
หรือไฟเลี้ยง");
```

```
    }
```

```
lastPowerCalcTime = millis();
```

```
}
```

```
void loop() {
```

```
    unsigned long currentMillis = millis();
```

```
    while (Serial1.available() > 0) {
```

```
        char c = Serial1.read();
```

```
        gps.encode(c);
```

```
        gpsByteCounter++;
```

```
    }
```

```
    if (gps.location.isValid() && gps.location.isUpdated()) {
```

```
        lastValidLat = gps.location.lat();
```

```
        lastValidLng = gps.location.lng();
```

```
    }
```

```
// อ่านค่าเซนเซอร์ความเร่งทุกๆ 1 วินาทีเพื่อเช็คพฤติกรรม
```

```
if (currentMillis - lastSensorReadTime >= 1000) {
```

```
    lastSensorReadTime = currentMillis;
```

```
    if (isGyroConnected) {
```

```
        sensors_event_t a, g, temp;
```

```
        if (mpu.getEvent(&a, &g, &temp)) {
```

```
            last_acc_x = a.acceleration.x;
```

```
            last_acc_y = a.acceleration.y;
```

```
            last_acc_z = a.acceleration.z;
```

```
            current_total_acc = sqrtf(powf(last_acc_x, 2.0f) + powf(last_acc_y, 2.0f) +  
powf(last_acc_z, 2.0f));
```

```
// 🚨 ตรวจสอบสลับโหมดอัตโนมัติ: ถ้ากระแสแรง (>25) หรือวิ่งขึ้นตระหนก (>15)
```

```

        if (current_total_acc > 25.0f || current_total_acc > 15.0f) {
            emergencyTriggerTime = currentMillis; // บันทึกเวลาที่เกิดเหตุ
        }
    }
}

// ตัดสินใจเลือกโหมดการทำงานและคาบเวลาส่ง
if (emergencyTriggerTime > 0 && (currentMillis - emergencyTriggerTime < 8000)) {
    // หากเกิดเหตุผิดปกติ จะตรึงโหมดฉุกเฉินไว้อย่างน้อย 8 วินาที
    loraSendInterval = 2000; // เร่งส่งถี่ๆ 2 วินาที
    currentModeStr = " 🚨 EMERGENCY (รอบส่ง 2s)";
} else {
    // สภาวะปกติ: ส่งทุกๆ 15 วินาที
    loraSendInterval = 15000;
    currentModeStr = " 🟢 NORMAL (รอบส่ง 15s)";
}

// ยิ่งแพ็กเกจ LoRa ตามช่วงเวลาที่โหมดกำหนด
if (currentMillis - lastLoRaSendTime >= loraSendInterval) {
    lastLoRaSendTime = currentMillis;

    // คำนวณหักลบประจุตามเวลาจริงก่อนส่ง
    updateVirtualBattery();

    packetIdCounter++;
    int gpsValidFlag = gps.location.isValid() ? 1 : 0;

    // โครงสร้างข้อความส่งออก: $PET,Lat,Lng,Accel,GpsFlag,PacketId,BatPct,RemHours (คั่นด้วย
    คอมมา 7 ตัว ครบ 8 ช่อง)

```

```
String payload = "$PET," + String(lastValidLat, 6) + ","
               + String(lastValidLng, 6) + ","
               + String(current_total_acc, 2) + ","
               + String(gpsValidFlag) + ","
               + String(packetIdCounter) + ","
               + String(batteryPercent) + ","
               + String(remainingHours, 1);
```

```
size_t bytesSent = LORA_SERIAL.println(payload);
```

```
int auxLiveState = digitalRead(LORA_AUX);
```

```
Serial.println("\n  ===== [ ลือครายงานสถานะภาคส่งปลอกคอ ESP32 ] =====");
```

```
Serial.print("  [LORA CHANNEL CONFIG: "); Serial.println(assignedChannelStr);
```

```
Serial.printf("  [OPERATION MODE]: %s\n", currentModeStr.c_str());
```

```
Serial.printf("  [LORA HARDWARE PULSE]: %s\n", (auxLiveState == HIGH) ? " 
```

```
ALIVE & READY" : "  BUSY / TRANSMITTING");
```

```
Serial.printf("  [PACKET SEQUENCE ID]: ส่งออกแพ็กเกจลำดับที่ #%lu\n",
packetIdCounter);
```

```
Serial.printf("  [BATTERY STATUS]: %d%% |  คงเหลือ: %.2f ชม. (ดึงกระแสเฉลี่ย: %.1f
mA)\n",
```

```
batteryPercent, remainingHours, currentBatDrainMA);
```

```
if (isGyroConnected) {
```

```
Serial.printf("  [MPU6050 HARDWARE STATUS]:  ONLINE (I2C SDA:%d,
SCL:%d)\n", I2C_SDA, I2C_SCL);
```

```
Serial.printf("  [MPU6050 TOTAL ACCEL]: %.2f m/s²\n", current_total_acc);
```

```
} else {
```

```
Serial.println("  [MPU6050 HARDWARE STATUS]:  OFFLINE / DISCONNECTED");
```

```

}

Serial.println("-----");

Serial.print(" 📄 ข้อความดิบที่ยิงออกอากาศ: "); Serial.println(payload);

Serial.printf(" ⚡ [ELECTRICAL OUT]: ไฟฟ้าพ่นออกจากพินสำเร็จ -> %zu ไบต์\n", bytesSent);

Serial.printf(" 📶 [NEO-GPS STREAM]: ข้อมูลดาวเทียมวิ่งเข้าชิป = %lu ไบต์ (Fix: %s)\n",
gpsByteCounter, gpsValidFlag ? "YES [ 🟢 ]" : "NO [ 🟡 ]");

Serial.println("=====");
}

if (currentMillis - lastDisplayTime >= 3000) {
    lastDisplayTime = currentMillis;

    Serial.printf(" 💡 [SYSTEM HEALTH]: โหมด %s | ส่งแล้ว: %lu แพ็กเกจ | Accel: %.2f m/s² |
Bat: %d%%\n",
                currentModeStr.c_str(), packetIdCounter, current_total_acc, batteryPercent);
}
}

```

LORA รั้ว

```
#define BLYNK_TEMPLATE_ID "TMPL6aEV5Tzyf"
```

```
#define BLYNK_TEMPLATE_NAME "Smart Pet Tracker"
```

```
#define BLYNK_AUTH_TOKEN "GFrp0mSW2vGnPnLvg-D1b_NbcksFxFzT"
```

```
#include <Arduino.h>
```

```
#include <WiFi.h>
```

```
#include <WiFiClient.h>
```

```
#include <WiFiClientSecure.h>
```

```
#include <BlynkSimpleEsp32.h>
```

```

const char* ssid = "WJNK";
const char* password = "12345678";

const String LINE_TOKEN =
"QCnqt656xzGsM3da7GlsgFd5R1otHqbVncg17unluhD9UvO0bzcS53m+tNhi+OcOVKEjUKf73iSE
NjbzZGjI6X9tKbZK4AYPb0o0kvQleYaPMZt2ySPNDwhDt6U1APj3ikBub5eVLZPgPyPb8DwWqgdB0
4t89/1O/w1cDnyilFU=";
const String LINE_USER_ID = "U7d93b01ef2609846c1efc716b4c492ca";

#define LORA_M0 22
#define LORA_M1 23
#define LORA_AUX 21

#define VPIN_GYRO V1
#define VPIN_MAP_TEXT V2
#define VPIN_STATUS V3 // ● รวมแสดงผล: [โหมด] + สถานะการเคลื่อนไหว + สถานะรั้ว
#define VPIN_SETHOME V4
#define VPIN_REFRESH V5 // ● ปุ่ม Quick Refresh สำหรับกดบังคับรีเฟรชพิกัดด่วน
#define VPIN_RADIUS V7
#define VPIN_IMPACT_LIMIT V8
#define VPIN_HOME_COORD V9
#define VPIN_CLEAR_HOME V10
#define VPIN_MAP_WEBLINK V11 // ● ตั้งค่า Webpage Button ให้ตรงกับขา V11 บนแอป
#define VPIN_BAT_LEVEL V12 // 📶 แลกระดับแบตเตอรี่ (Level H / V)
#define VPIN_BAT_REMAIN V13 // ⌚ เวลาคงเหลือ (Value Display)

double liveLat = 18.802500;
double liveLng = 98.951500;
float current_total_acc = 0.0;
bool isGpsValidFromCollar = false;

```

```
String receiverChannelStr = "UNKNOWN";
```

```
// 🪫 ตัวแปรเก็บสถานะแบตเตอรี่ที่รับมาจากปลอกคอ
```

```
int collarBatteryPercent = 100;
```

```
float collarRemainingHours = 5.0;
```

```
// ⚙️ ตัวแปรเก็บโหมดการทำงานของปลอกคอ
```

```
String currentCollarMode = "[โหมดปกติ - 15s]";
```

```
unsigned long lastPacketTime = 0;
```

```
unsigned long totalPacketsReceived = 0;
```

```
unsigned long lastReceivedPacketId = 0;
```

```
unsigned long totalMissedPackets = 0;
```

```
double homeLat = 0.0;
```

```
double homeLng = 0.0;
```

```
bool isHomeSet = false;
```

```
double maxAllowedRadius = 3.0;
```

```
float gyroThreshold = 15.0;
```

```
unsigned long lastDisplayTime = 0;
```

```
unsigned long lastBlynkStreamTime = 0;
```

```
unsigned long lastBlynkConnectAttempt = 0;
```

```
String motionStatus = "🟢 ปกติดี";
```

```
unsigned long highAccStartTime = 0;
```

```
bool isHighAccContinuous = false;
```

```
unsigned long lastImpactTime = 0;
```

```
void sendLineMessage(String message) {
```

```

if (WiFi.status() == WL_CONNECTED) {
    WiFiClientSecure client;
    client.setInsecure();
    if (client.connect("api.line.me", 443)) {
        String payload = "{\"to\":\"" + LINE_USER_ID +
        "\", \"messages\": [{\"type\": \"text\", \"text\": \"" + message + "\"}]}";
        client.println("POST /v2/bot/message/push HTTP/1.1");
        client.println("Host: api.line.me");
        client.println("Authorization: Bearer " + LINE_TOKEN);
        client.println("Content-Type: application/json");
        client.print("Content-Length: ");
        client.println(payload.length());
        client.println();
        client.println(payload);
        client.stop();
    }
}
}

```

```

double calculateDistance(double lat1, double lng1, double lat2, double lng2) {
    double dLat = (lat2 - lat1) * M_PI / 180.0;
    double dLng = (lng2 - lng1) * M_PI / 180.0;
    double a = sin(dLat / 2) * sin(dLat / 2) +
        cos(lat1 * M_PI / 180.0) * cos(lat2 * M_PI / 180.0) *
        sin(dLng / 2) * sin(dLng / 2);
    double c = 2 * atan2(sqrt(a), sqrt(1 - a));
    return 6371000.0 * c;
}

```

```

BLYNK_CONNECTED() {

```

```

    Blynk.syncVirtual(VPIN_RADIUS);
    Blynk.syncVirtual(VPIN_IMPACT_LIMIT);
}

BLYNK_WRITE(VPIN_RADIUS) { maxAllowedRadius = param.asDouble(); }
BLYNK_WRITE(VPIN_IMPACT_LIMIT) { gyroThreshold = param.asFloat(); }

BLYNK_WRITE(VPIN_SETHOME) {
    if (param.asInt() == 1) {
        homeLat = liveLat;
        homeLng = liveLng;
        isHomeSet = true;

        Blynk.virtualWrite(VPIN_STATUS, currentCollarMode + " 🏠 ตั้งพิกัด Home สำเร็จ!");
        Blynk.virtualWrite(VPIN_HOME_COORD, String(homeLat, 6) + "," + String(homeLng, 6));
    }
}

// 🟢 ปุ่ม Quick Refresh (V5) สำหรับกดบังคับรีเฟรชพิกัดด่วน
BLYNK_WRITE(VPIN_REFRESH) {
    if (param.asInt() == 1) {
        String mapLink = String(liveLat, 6) + "," + String(liveLng, 6);
        String dynamicWebLink = "https://maps.google.com/?q=" + mapLink + "&t=" +
String(millis());

        Blynk.setProperty(VPIN_MAP_WEBLINK, "url", dynamicWebLink);
        Blynk.virtualWrite(VPIN_MAP_TEXT, mapLink);

        Blynk.virtualWrite(VPIN_STATUS, currentCollarMode + " 🟢 บังคับอัปเดตลิงก์แผนที่ V11
แล้ว!");
    }
}

```

```

BLYNK_WRITE(VPIN_CLEAR_HOME) {
  if (param.asInt() == 1) {
    homeLat = 0.0;
    homeLng = 0.0;
    isHomeSet = false;

    Blynk.virtualWrite(VPIN_STATUS, currentCollarMode + " 🗑️ ล้างพิกัด Home เรียบร้อย!");
    Blynk.virtualWrite(VPIN_HOME_COORD, "0.000000,0.000000");
  }
}

```

```

void configureLoraReceiverChannel() {
  digitalWrite(LORA_M0, LOW);
  digitalWrite(LORA_M1, HIGH);
  delay(100);
  while(Serial2.available() > 0) Serial2.read();
  byte configCmd[] = {0xC0, 0x00, 0x06, 0x00, 0x00, 0x00, 0x62, 0x00, 0x49};
  Serial2.write(configCmd, 9);
  delay(200);
  if (Serial2.available() > 0 && Serial2.read() == 0xC1) {
    receiverChannelStr = "CH 73 (923.125MHz - 0x49) [ 🟢 ACTIVE]";
    delay(20);
    while(Serial2.available() > 0) { Serial2.read(); }
  } else {
    receiverChannelStr = "CH 73 (923.125MHz) [ 🟢 ACTIVE]";
  }
  digitalWrite(LORA_M0, LOW);
  digitalWrite(LORA_M1, LOW);
  delay(100);
}

```

```

void setup() {
    Serial.begin(115200);
    pinMode(LORA_M0, OUTPUT);
    pinMode(LORA_M1, OUTPUT);
    pinMode(LORA_AUX, INPUT);

    Serial2.begin(9600, SERIAL_8N1, 16, 17);
    configureLoraReceiverChannel();

    WiFi.setSleep(false);
    WiFi.begin(ssid, password);
    int timeout = 0;
    while (WiFi.status() != WL_CONNECTED && timeout < 8) { delay(500); timeout++; }
    Blynk.config(BLYNK_AUTH_TOKEN);
}

void loop() {
    unsigned long currentMillis = millis();

    if (WiFi.status() != WL_CONNECTED) {
        static unsigned long lastWifiRetry = 0;
        if (currentMillis - lastWifiRetry >= 10000) { lastWifiRetry = currentMillis; WiFi.begin(ssid,
password); }
    }

    if (WiFi.status() == WL_CONNECTED && !Blynk.connected()) {
        if (currentMillis - lastBlynkConnectAttempt >= 10000) { lastBlynkConnectAttempt =
currentMillis; Blynk.connect(1000); }
    }
}

```

```
if (Blynk.connected()) { Blynk.run(); }
```

```
if (Serial2.available() > 0) {
```

```
    String incomingString = Serial2.readStringUntil('\n');
```

```
    incomingString.trim();
```

```
    int petIndex = incomingString.indexOf("$PET");
```

```
    if (petIndex != -1) {
```

```
        String cleanPayload = incomingString.substring(petIndex);
```

```
        // ค้นหาตำแหน่ง Comma ทั้ง 7 ตำแหน่ง
```

```
        int c1 = cleanPayload.indexOf(',');
```

```
        int c2 = cleanPayload.indexOf(',', c1 + 1);
```

```
        int c3 = cleanPayload.indexOf(',', c2 + 1);
```

```
        int c4 = cleanPayload.indexOf(',', c3 + 1);
```

```
        int c5 = cleanPayload.indexOf(',', c4 + 1);
```

```
        int c6 = cleanPayload.indexOf(',', c5 + 1);
```

```
        int c7 = cleanPayload.indexOf(',', c6 + 1);
```

```
        if (c1 != -1 && c2 != -1 && c3 != -1 && c4 != -1 && c5 != -1) {
```

```
            lastPacketTime = currentMillis;
```

```
            totalPacketsReceived++;
```

```
            liveLat = cleanPayload.substring(c1 + 1, c2).toDouble();
```

```
            liveLng = cleanPayload.substring(c2 + 1, c3).toDouble();
```

```
            current_total_acc = cleanPayload.substring(c3 + 1, c4).toFloat();
```

```
            isGpsValidFromCollar = (cleanPayload.substring(c4 + 1, c5).toInt() == 1);
```

```
            unsigned long incomingPacketId = 0;
```

```

// ตรวจสอบสถานะ Battery
if (c6 != -1 && c7 != -1) {
    incomingPacketId = cleanPayload.substring(c5 + 1, c6).toInt();
    collarBatteryPercent = cleanPayload.substring(c6 + 1, c7).toInt();
    collarRemainingHours = cleanPayload.substring(c7 + 1).toFloat();
} else {
    incomingPacketId = cleanPayload.substring(c5 + 1).toInt();
}

if (lastReceivedPacketId > 0) {
    if (incomingPacketId > lastReceivedPacketId + 1) {
        unsigned long missed = incomingPacketId - lastReceivedPacketId - 1;
        totalMissedPackets += missed;
    }
}

lastReceivedPacketId = incomingPacketId;

// 🚨 ตรวจสอบสถานะการเคลื่อนไหวและกำหนดโหมดการทำงาน
if (current_total_acc > 25.0f) {
    currentCollarMode = "[ 🚨 โหมดฉุกเฉิน - 2s]";
    motionStatus = "💣 บาดเจ็บ/ล้มคว่ำ!";
    lastImpactTime = currentMillis;
    static unsigned long lastLineImpact = 0;
    if (currentMillis - lastLineImpact > 15000) {
        lastLineImpact = currentMillis;
        sendLineMessage(" 🚨 [Gateway LoRa] เตือนภัย! ตรวจพบแรงกระแทกหรือสัตว์
เลี้ยงล้มรุนแรงเฉียบพลัน!");
    }
} else if (current_total_acc > gyroThreshold) {
    if (highAccStartTime == 0) highAccStartTime = currentMillis;
}

```



```

        geoStatus = (distance > maxAllowedRadius) ? " 🚨 หนีเที่ยว! (" + String(distance, 1) +
" ม.)" : " 🏠 ในเขตรั้ว (" + String(distance, 1) + " ม.)";
        if (distance > maxAllowedRadius) {
            currentCollarMode = "[ 🚨 โหมดฉุกเฉิน - 2s]";
            static unsigned long lastGeoLineTime = 0;
            if (currentMillis - lastGeoLineTime > 30000) {
                lastGeoLineTime = currentMillis;
                sendLineMessage(" 🚨 [Gateway LoRa] สัตว์เลี้ยงแอบหนีเที่ยวออกจากขอบเขตบ้าน
แล้ว! พิกัด: https://maps.google.com/?q=" + mapLink);
            }
        }
    } else {
        geoStatus = " 📍 รอต้งโฮมทำงาน";
    }
}

```

// 1. อัปเดตพิกัดดิบเข้า V2

```
Blynk.virtualWrite(VPIN_MAP_TEXT, mapLink);
```

// 2. 🟢 อัปเดต URL ของ Webpage Button ขา V11

```
String dynamicWebLink = "https://maps.google.com/?q=" + mapLink + "&t=" +
String(currentMillis);
```

```
Blynk.setProperty(VPIN_MAP_WEBLINK, "url", dynamicWebLink);
```

// 3. 🎯 ส่งสถานะรวมที่ระบุโหมดชัดเจนเข้าขา V3

```
Blynk.virtualWrite(VPIN_STATUS, currentCollarMode + " " + motionStatus + " | " +
geoStatus);
```

// 4. 🔋 อัปเดตสถานะแบตเตอรี่ปลอกคอเข้า Widget บนแอป Blynk (V12 และ V13)

```
Blynk.virtualWrite(VPIN_BAT_LEVEL, collarBatteryPercent);
```

```

    Blynk.virtualWrite(VPIN_BAT_REMAIN, String(collarRemainingHours, 1) + " ชม.");
}

if (currentMillis - lastDisplayTime >= 2000) {
    lastDisplayTime = currentMillis;

    Serial.println("\n 📊 ===== [ ตรวจสอบร้ดตรวจสอบสถานะฐาน GATEWAY LOG ] =====");
    Serial.print(" 📡 [LORA CHANNEL CONFIG]: "); Serial.println(receiverChannelStr);
    Serial.printf(" ⚙️ [OPERATION MODE]: %s\n", currentCollarMode.c_str());
    Serial.printf(" 📶 [RF LINK STATUS]: %s (รับสำเร็จ = %lu แพ้กเกจ | หล่นหาย = %lu  

แพ้กเกจ)\n",
        (currentMillis - lastPacketTime < 18000 && lastPacketTime > 0) ? " 🟢
CONNECTED" : " 🟡 STANDBY...",
        totalPacketsReceived, totalMissedPackets);

    Serial.printf(" 📄 [LATEST ID]: ได้รับแพ้กเกจลำดับล่าสุด #%lu\n", lastReceivedPacketId);
    Serial.printf(" 🔋 [COLLAR BATTERY]: %d%% | ⏳ คงเหลือ: %.1f ชม.\n",
collarBatteryPercent, collarRemainingHours);

    Serial.printf(" 🧠 [MOTION]: %s | ⚡ แรงเร่ง = %.2f m/s2\n", motionStatus.c_str(),
current_total_acc);

    Serial.printf(" 📍 [LIVE COORD]: Lat=%.6f, Lng=%.6f (Fix: %s)\n", liveLat, liveLng,
isGpsValidFromCollar ? "YES" : "NO");

    Serial.println("=====");
}
}

```